



On the classification of fog computing applications: A machine learning perspective

Judy C. Guevara^a, Ricardo da S. Torres^b, Nelson L.S. da Fonseca^{a,*}

^a Institute of Computing, University of Campinas, Campinas, 13083-852, SP, Brazil

^b Department of ICT and Natural Sciences, Norwegian University of Science and Technology (NTNU), Ålesund, Norway

ARTICLE INFO

Keywords:

Fog computing
Edge computing
Cloud computing
Internet of things
Scheduling
Classes of service
Quality of service
Machine learning
Feature selection
Attribute noise
Classification algorithms

ABSTRACT

Currently, Internet applications running on mobile devices generate a massive amount of data that can be transmitted to a Cloud for processing. However, one fundamental limitation of a Cloud is the connectivity with end devices. Fog computing overcomes this limitation and supports the requirements of time-sensitive applications by distributing computation, communication, and storage services along the Cloud to Things (C2T) continuum, empowering potential new applications, such as smart cities, augmented reality (AR), and virtual reality (VR). However, the adoption of Fog-based computational resources and their integration with the Cloud introduces new challenges in resource management, which requires the implementation of new strategies to guarantee compliance with the quality of service (QoS) requirements of applications.

In this context, one major question is how to map the QoS requirements of applications on Fog and Cloud resources. One possible approach is to discriminate the applications arriving at the Fog into Classes of Service (CoS). This paper thus introduces a set of CoS for Fog applications which includes, the QoS requirements that best characterize these Fog applications. Moreover, this paper proposes the implementation of a typical machine learning classification methodology to discriminate Fog computing applications as a function of their QoS requirements. Furthermore, the application of this methodology is illustrated in the assessment of classifiers in terms of efficiency, accuracy, and robustness to noise. The adoption of a methodology for machine learning-based classification constitutes a first step towards the definition of QoS provisioning mechanisms in Fog computing. Moreover, classifying Fog computing applications can facilitate the decision-making process for Fog scheduler.

1. Introduction

Cloud computing enables ubiquitous access to shared pools of configurable resources and services over the Internet that can be rapidly provisioned with minimal management effort (Mell and Grance, 2011). However, with the increasing relevance of the Internet of Things (IoT), mobile and multimedia applications, the transfer delays between the Cloud and an end device have been deemed too long and not suitable for latency-sensitive applications, making the main limitation in the use of the Cloud (OpenFog Reference Architecture, 2017; Alkassab et al., 2017) for latency-sensitive and mobile applications (Bittencourt et al., 2017; Hu et al., 2017; Kumari et al., 2019).

The plethora of applications running on the Internet has heterogeneous processing and communications demands, as well as Quality of Service (QoS) requirements. While multimedia demand processing

power and storage space (Byers, 2017), others such as mission-critical require strict response time. Mobile users need to have continuous access to applications when on the move. Moreover, IoT devices and sensors generate large amounts of data. Not all data need to be sent to the Cloud while some data have to be processed immediately.

Fog computing aims at coping with these demands by hosting Cloud services on connected heterogeneous devices, typically, but not exclusively located at the edge of the network (Bonomi et al., 2012; Wang et al., 2019). The Fog provides a geographically distributed architecture for computation, communication, and storage, which targets real-time applications and mobile services. End-users benefit from pre-processing of workloads, geo-distribution of resources, low latency responses, device heterogeneity (Bonomi et al., 2012), and location/content awareness (Deng et al., 2016). The Fog can support the diversity of applications requirements in the Cloud to Things (C2T)

* Corresponding author.

E-mail addresses: jguevara@lrc.ic.unicamp.br (J.C. Guevara), ricardo.torres@ntnu.no (R.S. Torres), nfonseca@ic.unicamp.br (N.L.S. da Fonseca).

<https://doi.org/10.1016/j.jnca.2020.102596>

Received 7 May 2019; Received in revised form 7 February 2020; Accepted 9 March 2020

Available online 17 March 2020

1084-8045/© 2020 Elsevier Ltd. All rights reserved.

continuum, which is comprised of end devices, one or more levels of Fog nodes, and the Cloud. Fog nodes located at the edge of the network are usually limited in resources (Shao et al., 2019). Still, their use involves only brief delays in communication while the Cloud has a large (“unlimited”) number of resources, but involves long delays in communication. On the lowest level of this continuum, the initial processing can be carried out, and results passed on to a higher layer in a Fog hierarchy, or to the Cloud itself for further processing (Byers, 2017; Arkian et al., 2017).

Applications are usually composed of (dependent) tasks. The scheduling of tasks using C2T resources is much more challenging than that of tasks on grids (confined systems) (Batista and Fonseca, 2010; Batista and da Fonseca, 2011; Krauter et al., 2002; Xu et al., 2011) and on Clouds (more homogeneous systems) (Bittencourt et al., 2012; Fonseca and Boutaba, 2015; Genez et al., 2019; Tsai and Rodrigues, 2014; Kliazovich et al., 2016; Wu et al., 2015) due to the considerable heterogeneity of both demands of applications and the capacity of the devices. Consequently, there is a need for schedulers to analyze various parameters before making decisions as to where tasks and virtualized resources should be run, including consideration of the availability of resources and their cost (Byers, 2017).

It is thus crucial for the efficient provisioning of services that the demands of applications arriving at the edge of the network be well understood and classified so that resources can be assigned for their processing. The mapping of applications onto Class of Services (CoS) should facilitate the matching between task requirements and resources, since labeling these tasks removes the burden of the analysis of the application requirements by the scheduler. Without a precise classification, the scheduling of application tasks and the allocation of resources can be less than optimal due to the complexity in dealing with the diversity of QoS and resource requirements. Mapping applications onto Class of Service is typical in communications network technologies that support QoS, such as LTE, 5G (Ali et al., 2017), and ATM (Cohen et al., 1998) networks, and it is a key element for network providers to be able to offer different grades of service. Moreover, it is essential for the use of efficient classification algorithms to deal with the specific characteristics of C2T.

Techniques for the classification of network traffic have been extensively studied for the past few decades, especially for the provisioning of secure network services (Callado et al., 2009; Finsterbusch et al., 2014). However, very little attention has been paid to the classification of the demands and requirements of applications and services over the Internet (Zhong et al., 2004). Moreover, device mobility and the IoT have introduced new applications to the Internet, and these applications have not been considered in any of the classification schemes. This paper contributes with the definition of a set of Classes of Service for Fog computing, that takes into consideration the QoS requirements of the most relevant Fog applications, thus allowing the differentiation of the demands of a broad spectrum of applications.

The original contribution of this paper is a methodology for the classification of applications as to Class of Service, which considers their QoS requirements. This methodology can be used to design effective classifiers with an output that can facilitate the job of schedulers of applications tasks. In the scenario assumed in this paper, users subscribe directly or indirectly to Fog infrastructure services. The first packet of a flow contains the QoS requirements of the application generating the packet flow. The proposed classifier will then map this application into a CoS using the information provided in the first packet. The CoS can then be used by a Fog task scheduler to schedule application tasks and allocate resource to these tasks. It is our best knowledge that no previous paper has addressed the classification of Fog applications. A case study dealing with the classification of a dataset containing Fog application features illustrates the use of this methodology. This methodology constitutes a first step towards the definition of a QoS provisioning framework to facilitate the definition of new business models in Fog computing.

The rest of this paper is structured as follows. Section 2 overviews related work. Section 3 proposes a set of classes of service for Fog computing, and provides a mapping between the recommended classes of service and the layers of the reference architecture presented by the OpenFog Consortium. Section 4 introduces the use of a typical machine learning-based methodology for Fog computing. Section 5 illustrates the implementation of the methodology described in Section 4 with a case study, which includes two scenarios, differing by the place where noise is introduced (either in the training set or in the testing set). Finally, Section 6 concludes the paper and points out directions for future research.

2. Related work

Different studies have analyzed application requirements to develop service models for both Cloud and Fog computing. In Cloud computing, these studies have focused on Service Level Agreements (SLA) (Alhamad et al., 2010; Wu et al., 2013; Emeakaro et al., 2010, 2012) and Quality of Experience (QoE) management (Hobfeld et al., 2012), while in Fog computing, investigations have emphasized processing and analytics for specific applications (Yang, 2017), scheduling of applications to resources (Cardellini et al., 2015), resource estimation (Aazam et al., 2016) and allocation (Wang et al., 2017; He et al., 2018) for the processing of applications, and service placement (Mahmud et al., 2019; Skarlat et al., 2017). Next, these studies are briefly described.

Alhamad et al. (2010) presented nonfunctional requirements of Cloud consumers, and defined the most important criteria for the definition and negotiation of SLAs between consumers and Cloud service providers.

Wu et al. (2013) developed a Software as a Service (SaaS) broker for SLA negotiation. The aim was to achieve the required service efficiently when negotiating with multiple providers. The proposal involved the design of counter offer strategies and decision-making heuristics which considered time, market constraints and trade-off between QoS parameters. Results demonstrated that the proposed approach increases by 50% the profit and by 60% the customer satisfaction level.

Emeakaro et al. (2010) presented an approach for mapping SLA requirements to resource availability, called LoM2Hi, which is capable of detecting future SLA violations based on predefined thresholds to avert these violations.

Emeakaro et al. (2012) proposed an architecture for application monitoring architecture, named Cloud Application SLA Violation Detection architecture (CASViD). CASViD monitors and detects SLA violations at the application layer, and includes tools for resource allocation, scheduling, and deployment. Results showed that the proposed architecture is efficient in monitoring and detecting situations of a single SLA violation.

Hobfeld et al. (2012) discussed the challenges for QoE provisioning for Cloud applications with emphasis on multimedia applications. The authors also presented a QoE-based classification scheme of Cloud applications aligned to the end-user experience and usage domain.

Yang (2017) investigated common components of IoT systems such as stream analytics, event monitoring, networked control, and real-time mobile crowdsourcing, for defining an architecture for Fog data streaming.

Cardellini et al. (2015) modified the Storm data stream processing system (DSP) to operate in a geographically distributed and highly variable environment. To demonstrate the effectiveness of the extended Storm system, the authors implemented a distributed QoS aware scheduling algorithm for placing DSP applications near to the data sources and the final consumers. The main limitation of this study is the instability of the scheduling algorithm that affects negatively the application availability. Results showed that the distributed QoS-aware scheduler outperforms the default centralized one, improving the application performance.

Aazam et al. (2016) developed a method, called Media Fog Resource Estimation (MeFoRE), to provide resource estimation on the basis of service give-up ratio, the record of resource usage and the required quality of service. The aim was to avoid resource underutilization and enhance QoS provisioning. MeFoRE methodology uses real IoT traces and traces of Amazon EC2 service.

Wang et al. (2017) presented an edge architecture, called mobile micro-Cloud, to provide situational awareness to processing elements. The authors introduced an approach for consistent representation of application requirements for deployment in the mobile micro-Cloud environment.

He et al. (2018) introduced QoE model which included user oriented metrics such as the Mean Opinion Score (MOS) and content popularity as well as the cost of cache allocation and transmission rate. The computed QoE value was used in a resource allocation problem formulated as a maximization problem solved by a shortest path tree algorithm. Results showed the benefit of using dynamic allocation to achieve high QoE values.

Mahmud et al. (2019) proposed a QoE-aware application placement policy that prioritizes placement requests according to user expectation and the Fog available capacity. Two fuzzy logic models were employed to map applications to resources. Requests for application placement consider metrics such as service access rate, required resources and expected processing time. A linear optimization problem ensures that prioritized requests for application placement are mapped to Fog resources so that user QoE is maximized. Results indicated that the policy significantly reduces the processing time, resource availability, and the quality of service.

Skarlat et al. (2017) evaluated the placement of IoT services on Fog resources, taking into account QoS requirements. The authors proposed an approach for the optimal sharing of resources among IoT services by employing a formal model for Fog systems. The authors introduced the Fog Service Placement Problem (FSPP) for placing IoT services on virtualized Fog resources while taking into account constraints such as execution time deadlines. Results showed that the proposed optimization model prevents QoS violations and decreases the execution cost when compared to a purely Cloud-based solution.

The classification methodology introduced in this paper differs from the aforementioned proposals by the definition of a set of Class of Service for Fog computing and the use of machine learning algorithms to map applications onto these classes. To our knowledge, this is the first study that introduces a machine learning classification methodology to discriminate Fog computing applications on the basis of QoS requirements. It is crucial for the efficient provisioning of services that the demands of applications arriving at the edge of the network be classified so that resources can be assigned for their processing. The related work in Fog computing reported above concentrates on resource allocation and scheduling of applications. Most of the decisions on resource allocation and scheduling in those papers is limited to information on resource consumption by the applications. They do not consider several QoS requirements as done in the present manuscript. Moreover, no previous paper has addressed the classification of applications on the Cloud to Things (C2T) continuum. Most of the work dealing with SLAs and QoS/QoE considers only the Cloud. The Fog layers in C2T will increase the capacity of the system to support new applications, especially those with real-time constraints, which are not possible to be handled by the Cloud.

3. Classes of Service for Fog applications

Fog computing enables new applications, especially those with strict latency constraints and those involving mobility. These new applications will have heterogeneous QoS requirements and will demand Fog management mechanisms to cope efficiently with that heterogeneity. Thus, resource management in Fog computing is quite challenging, calling for integrated mechanisms capable of dynamically adapting the

allocation of resources. A very first step in resource management is to separate incoming flow of requests into Classes of Service (CoS) according to their QoS requirements.

Bandwidth. Some applications request a minimally guaranteed throughput, i.e., a Guaranteed Bit Rate (GBR). Multimedia applications are bandwidth sensitive, although some of them use adaptive coding techniques to encode digitized voice or video at a rate that matches the currently available bandwidth.

Delay sensitivity. Some applications involve a specific latency threshold, below which latency must be assured, especially for real-time applications.

Loss sensitivity indicates the proportion of packets which does not reach their destination.

Reliability is concerned with the ability of the Fog components to carry out the desired operation in the presence of many types of failure. Some applications need to have failed Fog components quickly reestablished so that tasks can be performed within some latency bounds.

Availability provides a measure of how often the resources of the Fog are accessible to end-users. High availability is needed by applications and services that must be running all the time, such as mission-critical applications.

Security refers to the design and implementation of authentication and authorization techniques to protect personal and critical information generated by end users.

Data location indicates where the application data should be stored. Data can be stored locally, at the end device itself; near, or at a Fog node, or in a remote repository, in the Cloud. Requirements of data location for an application depend on factors such as response time constraints, the computational capacity of each Fog layer, and available capacity on network links.

Mobility is an intrinsic characteristic of many edge devices. Continuity of the offered services should be ensured, even for highly mobile end-users. Continuous connectivity is essential for the processing needed.

Scalability is related to the capability of an application to operate efficiently, even in the presence of an increasing number of requests from end users. The number of users in a Fog can fluctuate due to the mobility of the users, as well as the activation of applications or sensors. Streams of data in big data processing may need to be processed within a specific time frame. The demand on Fog nodes can fluctuate and resource elasticity needs to be provided to cope with these demands.

The mapping of applications into a set of classes of service is the first step in the creation of a resource management system capable of coping with the heterogeneity of Fog applications. This paper proposes various classes of service for Fog computing: Mission-critical, Real-time, Interactive, Conversational, Streaming, CPU-bound, and Best-effort. These classes will be defined and the typical applications using these classes identified.

The first CoS to be discussed is the *Mission-critical (MC)* class. It comprises applications with a low event to action time-bound, regulatory compliance, military-grade security, privacy, and applications in which a component failure would cause a significant increase in the safety risk for people and the environment. Applications include healthcare and hospital systems, medical localization, healthcare robotics, criminal justice, drone operations, industrial control, financial transactions, ATM banking systems, and military and emergency operations.

The *Real-time (RT)* class, on the other hand, groups applications requiring tight timing constraints in conjunction with effective data delivery. In this case, the speed of response in real-time applications is critical, since data are processed at the same time they are generated. In addition to being delay sensitive, real-time applications often require a minimum transmission rate and can tolerate a certain amount of data loss. This real-time class includes applications such as online gaming, virtual reality, and augmented reality.

The third class is denominated *Interactive (IN)*. In this case, responsiveness is critical, the time between when the user requests and actions manifested at the client being less than a few seconds. Moreover, users of interactive applications can be end devices or individuals. Examples of applications belonging to this class are interactive television, web browsing, database retrieval, server access, automatic database inquiries by tele-machines, pooling for measurement collection, and some IoT deployments.

The fourth class is the *Conversational (CO)* class. These applications include some of the video and Voice-over-IP (VoIP). They are characterized by being delay-sensitive but loss-tolerant with delays less than 150 ms being perceived by humans, delays between 150 and 400 ms can be acceptable, and those exceeding 400 ms resulting in completely unintelligible voice conversations. On the other hand, conversational multimedia applications are loss-tolerant with occasional losses causing only occasional glitches in audio or video playback, and these losses can often be partially or fully concealed (Kurose and Ross, 2012).

The fifth class of service is *Streaming (ST)*, which releases the user to download entire files, although in potentially long delays are incurred, before playout begins. Streaming applications are accessed by users on demand and must guarantee interactivity and continuous playout to the user. For this reason, the most critical performance measure for streaming video is average throughput (Kurose and Ross, 2012). Additionally, streaming can refer to stored or live content. In both cases, the network must provide each flow with an average throughput that is larger than the content consumption rate. In live transmissions, the delay can also be an issue, although the timing constraints are much less stringent than those of conversational voice. Thus, delays of up to 10 s or so from when the user chooses to view a live transmission to when playout begins can be tolerated. Examples of streaming applications are high-definition movies, video (one-way), streaming music, and live radio and television transmissions.

The sixth class is *CPU-Bound (CB)* class which is used by applications involving complex processing models, such as those in decision making, which may demand hours, days, or even months of processing. Face recognition, animation rendering, speech processing, and distributed camera networks, are examples of CPU-Bound applications.

The final class is that of *Best-Effort (BE)*. It is dedicated to traditional best-effort applications over the Internet. For Best-effort applications, long delays are annoying but not particularly harmful; the completeness and integrity of the transferred data, however, are of paramount importance. Some examples of the Best-Effort class are e-mail downloads, chats, SMS delivery, FTP, P2P file sharing, and M2M communication.

Table 1 presents the relationship between the applications supported by Fog computing and the requirements of the classes of service explained above. The first column shows the recommended priority level of each class for potential adoption in scheduling systems.

Table 2 shows the range of QoS requirement values for each class of service: Bandwidth (Kurose and Ross, 2012; Hobfeld et al., 2012), Reliability (Böhmer et al., 2011), Security (Khan et al., 2017), Data storage (Alhamad et al., 2010; Hobfeld et al., 2012), Data location (Alhamad et al., 2010; Hobfeld et al., 2012; Böhmer et al., 2011), Mobility (Böhmer et al., 2011), Scalability (Alhamad et al., 2010; Hobfeld et al., 2012; Böhmer et al., 2011), Delay sensitivity (Byers, 2017; Hobfeld et al., 2012), Loss sensitivity (Ali et al., 2013). These ranges are used to generate the synthetic dataset of Fog applications and are employed for training and testing samples to evaluate the classifiers in this paper (Section 5).

The reference architecture proposed by the OpenFog consortium in (OpenFog Reference Architecture, 2017) provides a structural model for Fog-based computation on several tiers of nodes. The tiers differ in relation to the amount and type of work which can be processed on them, the number of sensors, the capacity of the nodes, the latency between nodes, reliability, and availability of nodes. Nodes at the edge are involved in sensor data acquisition/collection, data normalization,

and command/control of sensors and actuators, while nodes that are closer to the Cloud aggregate and transform data into knowledge. As one moves further away from the edge, the overall intelligence and capacity of the system increase.

Fig. 1 presents a distributed multi-layer architecture, based on the OpenFog reference architecture, which is composed of four layers: the Cloud, at the top, a layer of end devices at the bottom and two intermediate Fog layers. Fig. 1 also provides a mapping between the proposed classes of service and a multi-layer Fog-Cloud architecture. The bottom layer, composed of IoT and end-devices, sends application requests to the classifier, located on the first Fog layer. An application request is composed of the workflow of tasks and their demands, as well as the QoS requirements for the application. The classifier identifies the CoS of the application and forwards it to the scheduler, which decides where the application should be processed, whether on the first Fog layer, on the second Fog layer, or in the Cloud.

Not all layers are involved in the processing of all tasks. Since Real-time, Interactive, Conversational, and Streaming applications, such as online sensing, object hyperlinking, video conferencing, and stored streaming are delay-sensitive, these applications must be processed as close as possible to the end user, preferably at nodes located on the first and second Fog layer. CPU-bound applications require many processing resources, and for this reason, can involve all the layers of the reference architecture for the processing of tasks. Best-effort applications, such as e-mails, can be processed in the Cloud since there are no delay constraints for this class.

The possibility of having a hierarchical layered system is one of the significant differences between Fog computing and edge computing. Edge computing is mainly concerned in bringing the computation facilities closer to the user; however, in a flat non-hierarchical architecture (Mahmud and Buyya, 2016). A layered architecture can introduce additional communication overhead for processing tasks at different layers. However, it has been shown that if the scheduling of tasks and resource reservations are properly carried out, processing in a hierarchical architecture can reduce communication latency and task waiting time for processing when compared to a flat architecture (Chekired et al., 2018).

In the scenario assumed in this paper, users subscribe directly or indirectly to Fog infrastructure services. The first packet of a flow contains the QoS requirements of the application generating the packet flow. The proposed classifier will then map this application into a CoS using the information provided in the first packet. Alternatively, the first packet could already carry the CoS of the application. However, such an option would make rigid the CoS adopted by the Fog provider, preventing the redefinition of this CoS for the handling of new applications with unique QoS requirements.

4. Classification methodology based on machine learning

This section introduces a methodology for choosing and evaluating classifiers for Fog computing applications. It provides a step-by-step procedure for grounded choices of classifiers. Indeed, this methodology can be easily modified for the classification of applications in networked systems.

Classification techniques based on ML aim at mapping a set of new input data to a set of discrete or continuous valued output. Fig. 2 summarizes the key steps in the building of a classifier of Fog applications based on ML algorithms. In this paper, the classification steps were executed offline. Indeed, the best performing classifier evaluated in these steps can be executed on-line in an operational Fog.

The first step is the creation of a *labeled dataset* containing QoS attributes of Fog applications, which can be either real or synthetic. A real dataset is one collected from a system in operation while a synthetic one involves data collection generated by models. Real-world datasets usually contain sensitive data (McGregor et al., 2004) and are often unavailable for the maintenance of the user information. Thus, the use

Table 1
Class of Service and their requirements.

Allocation Priority	Class of Service	Service Quality Requirements																		Applications			
		Bandwidth	Reliability			Security			Data storage			Data location			Mobility			Scalability			Delay sensitivity	Loss sensitivity	
			Low	Important	Critical	Low	Medium	High	Transient	Short duration	Long duration	Local	Vicinity	Remote	Low	Medium	High	Low	Medium				High
1	MC	GBR			✓			✓	✓			✓				✓	✓	✓		✓	Yes	Yes	Healthcare, criminal justice, financial, biological traits, residence and geographic, military, emergency. Online gaming, IoT deployments, industrial control, virtual and augmented reality, interactive television, telemetry. Interactive television, object hyperlinking, web browsing, database retrieval, server access, automatic database enquiries by tele-machines, pooling for measurements collection, and some IoT deployments. Voice messaging, VoIP, videoconference. Internet radio, video (one-way), high quality streaming audio. Face recognition, animation rendering, speech processing, distributed camera networks. Network signaling, all non-critical traffic such as TCP-based data: www, e-mail, chat, FTP, P2P file sharing, progressive video and other miscellaneous traffic.
2	RT	GBR		✓			✓		✓			✓				✓			✓		Yes	No	
3	IN	GBR	✓					✓	✓			✓				✓				✓	Yes	Yes	
4	CO	GBR		✓			✓		✓			✓						✓		✓	Yes	No	
5	ST	GBR		✓			✓				✓		✓			✓	✓		✓		Yes	No	
6	CB	GBR		✓				✓		✓	✓			✓		✓			✓		Yes	Yes	
7	BE	NGBR	✓			✓					✓			✓		✓				✓	No	Yes	

Table 2
Intervals of the QoS requirements for Fog computing.

QoS Requirements	Nominal Categories	Intervals	Class of Service						
			MC	RT	IN	CO	ST	CB	BE
Bandwidth (Mbps)	Low	$0 < x \leq 1$	✓		✓	✓			✓
	Medium	$1 < x \leq 5$					✓	✓	
	High	$5 < x \leq 1000$		✓					
Reliability	Low	$x = 1$			✓				✓
	Important	$x = 2$		✓		✓	✓	✓	
	Critical	$x = 3$	✓						
Security	Low	$x = 1$							✓
	Medium	$x = 2$		✓		✓	✓		
	High	$x = 3$	✓		✓			✓	
Data storage (h)	Transient	$0 < x \leq 1$	✓	✓	✓	✓			
	Short duration	$1 < x \leq 730$						✓	
	Long duration	$730 < x \leq 8760$					✓	✓	✓
Data location (ms)	Local	$0 < x \leq 10$	✓	✓					
	Vicinity	$10 < x \leq 20$			✓	✓			
	Remote	$20 < x \leq 100$					✓	✓	✓
Mobility (Km/h)	Low	$0 < x \leq 5$	✓	✓	✓			✓	✓
	Medium	$5 < x \leq 25$	✓				✓		
	High	$25 < x \leq 100$	✓			✓	✓		
Scalability (No. of IoT users/end users)	Low	$0 < x \leq 60$		✓					
	Medium	$60 < x \leq 120$					✓	✓	
	High	$120 < x \leq 200$	✓		✓	✓			✓
Delay sensitivity (Interaction latency in ms)	Low	$1000 < x \leq 100000$					✓	✓	✓
	Moderate	$10 < x \leq 1000$			✓	✓			
	High	$0 < x \leq 10$	✓	✓					
Loss sensitivity (PELR)	Low	$10^{-3} < x \leq 10^{-2}$				✓			
	Moderate	$10^{-6} < x \leq 10^{-3}$		✓			✓		
	High	$0 < x \leq 10^{-6}$	✓		✓			✓	✓

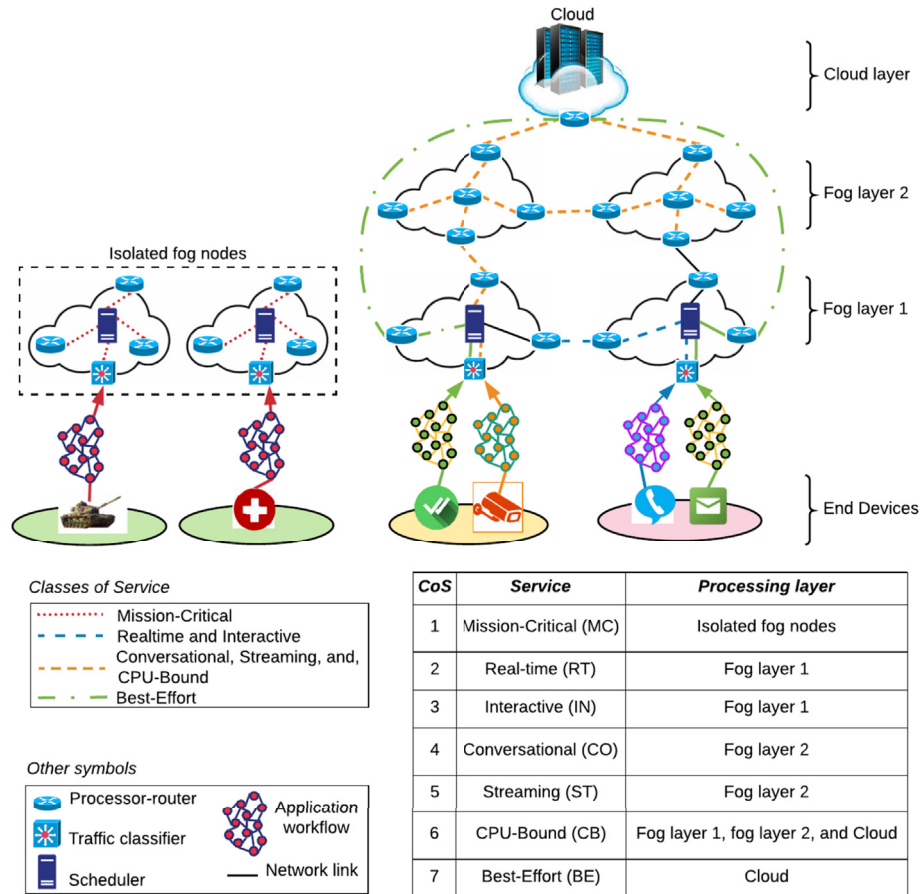


Fig. 1. Fog computing architecture and Class of Service.

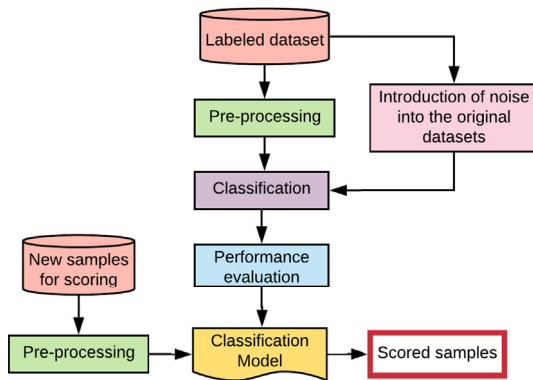


Fig. 2. Typical ML-based classification methodology adopted in the context of Fog computing networks.

of synthetic data sets is quite common, especially in the studies of systems yet to be built.

Since the value of QoS attributes differs widely, these values should be *pre-processed* to produce compatible ranges of values for classification. Pre-processing includes operations for data transformation, which standardize and consolidate data into more appropriate forms for classification, while data reduction includes the selection and extraction of both features and examples in a database (García et al., 2015; Tan et al., 2005). Data normalization avoids the handling of an attribute which has large values that dominate the results of the classification, thus improving the predictive power of the model. Feature selection, on the other hand, removes redundant and irrelevant data from the input of the classifier, without compromising critical attribute information (Tan et al., 2005).

Noise is an unavoidable problem in collecting data from real-world systems. It can change the knowledge extracted from the data set and affects the accuracy of the classification, building time, size, and interpretability of the classifier (Zhong et al., 2004; Zhu and Wu, 2004). Common sources of noise are channel capacity fluctuation, fluctuation in the availability of computational resources, imprecision inherited from measurement tools, and the inability to accurately estimate the true demands of applications.

In such noisy scenarios, robustness is considered more important than performance because robustness allows a priori knowledge of the behavior expected from a learning method despite noise when it is unknown (García et al., 2015). Robustness (Huber, 1981) is defined as the capability of an algorithm to be insensitive to data corruption and, consequently, more resilient against the impact of noise. A copy of the original dataset should be contaminated by the *introduction of noise* at different levels to check the robustness of a classifier. In this paper, uniform attribute noise levels of 10%, 30%, and 50% are employed. The performance of the classifiers which learned from the original data set is compared to that of those which learned from a noisy data set. The most robust classifiers are those which learned from noisy data sets yet produced results similar to those learned from a noise-free data set (García et al., 2015).

Classification techniques based on ML can then be applied. The performance of the classifier should be assessed by a *performance evaluation process*, which encompasses both the measurement of performance and the result of statistical tests. The adequacy of performance is usually assessed by metrics such as accuracy, efficiency, and robustness. Statistical testing gathers evidence of the extent to which an evaluation metric on the resampled data sets is representative of the general behavior of the classifier (Naqa et al., 2015).

At this point, the *classification model* is ready to receive new input for scoring. The new data, however, must also be subjected to a *pre-processing* process.

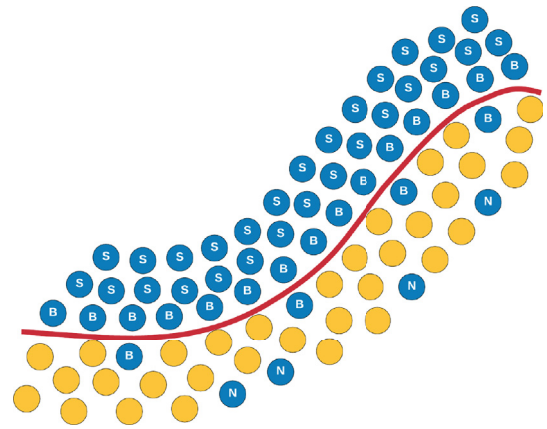


Fig. 3. The three types of samples considered in this paper: safe (S), borderline (B), and noisy (N).

5. Classification of Fog applications

This section illustrates step by step the methodology presented in the previous section for the classification of Fog applications using the CoS presented previously. Moreover, an example of a Decision Tree that classifies Fog computing applications from the values of their QoS requirements is provided at the end of this section.

5.1. Labeled dataset

To train and test the classifiers employed in this paper, we built a dataset¹ composed of 14,000 mutually exclusive applications generated from data in the intervals of values acceptable for each QoS requirement of the application. 90% of the data were reserved for training, while the remaining 10% were used for testing. It was assumed that each incoming application had additional fields containing nine QoS requirements, from now on referred to as “attributes”: Bandwidth, Reliability, Security, Data Storage, Data location, Mobility, Scalability, Delay sensitivity, and Loss sensitivity.

Attribute values were assigned by employing a uniform probability distribution, within the intervals specified for each CoS in Table 2. An independent random number generator randomly created the values of each attribute. Transient data were removed according to the Moving Average of Independent Replications procedure (Jain et al., 1991). Attribute values were made up of safe and borderline examples. Safe examples were placed in relatively homogeneous areas concerning the class label. Borderline examples, on the other hand, are located in the area surrounding class boundaries, where different classes overlap. Also, to estimate the robustness of the classifiers, the third group of attribute values, called noisy examples, was generated. The term noisy sample will be used in this paper to refer to the samples generated to represent the corruption of their attribute values.

Fig. 3 illustrates the safe samples, labeled as S, the borderline examples, labeled as B, and the noisy samples, labeled as N. The continuous line shows the decision boundary between the two classes.

5.2. Pre-processing

Z-score normalization was used to adjust attribute values defined on a different scale. Mean and standard deviation were computed on the training set, and then, the same mean and standard deviation were then used to normalize the testing set.

We reduced the number of input attributes to be used by classification algorithms. This process, known as dimensionality reduction,

¹ publicly available at <http://bit.ly/34x6X10>.

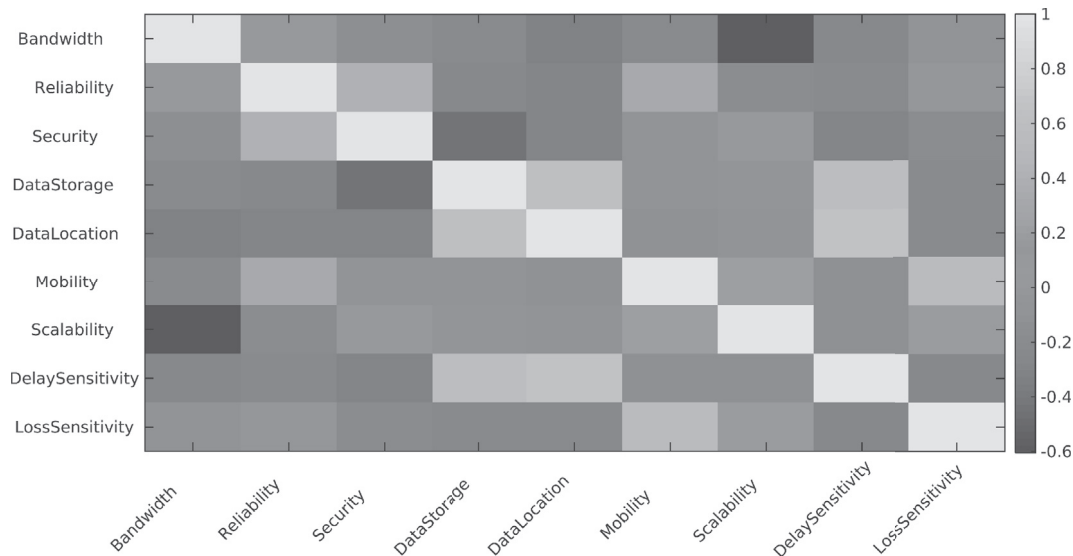


Fig. 4. Attribute association estimates.

removes irrelevant, redundant, and noisy information from the data, often leading to enhanced performance in learning and classification tasks (Roffo, 2016). Two techniques can be used for dimensionality reduction: one, by using feature selection techniques such as Relief-F (Liu and Motoda, 2007), CFS (Guyon et al., 2002), MCFS (Cai et al., 2010), and the Student's t-test, which rank the given feature set so that the least significant features can be removed from the problem. The second way involves feature extraction techniques, such as Principal Components Analysis (PCA), which creates new features from the given feature set. The resulting number of the features is less than that the initial set of features.

In this paper, the significance level of the impact of each input attribute on the system is determined utilizing a PCA, guided further by a correlation analysis and the semantics of the Fog computing environment. The correlation analysis and complementarity with the PCA are explained below.

The correlation analysis is a statistical evaluation technique used to study the strength of a dependence between pairs of attributes. Fig. 4 provides a graphic representation of the correlation matrix among dataset attributes for Fog applications.

The correlation matrix shows that there is a statistical association of more than 50% between the following variables: “Data storage” and “Data location” (0.623), “Data storage” and “Delay sensitivity” (0.596), “Loss sensitivity” and “Mobility” (0.563), “Bandwidth” and “Scalability” (−0.605) and, “Data location” and “Delay sensitivity” (0.674). The symbol “−” in the correlation value between the attributes of “Bandwidth” and “Scalability” indicates an inverse relationship between the two.

Principal components analysis (PCA) (Pearson, 1901) is a common approach for dimensionality reduction that uses techniques from linear algebra to find new attributes, denominated principal components, which are linear combinations of the original attributes. They are orthogonal to each other, and capture the maximum amount of variation in the data. Fig. 5 shows the scree plot of the percent variability explained by each principal component.

As illustrated by Fig. 5, the first seven principal components explain 94.314% of the total variance. The first component by itself explained less than 35% of the variance, so more components might be needed. Also, Fig. 5 reveals that the first three principal components explain roughly two-thirds of the total variability in the standardized ratings.

In addition to the percent variability explained by each principal component, all nine attributes were represented in a bi-plot by a vector. The direction and length of the vector indicate the contribution of

each attribute to the two principal components in the plot. For instance, Fig. 6 shows the coefficients of each attribute concerning the first two principal components.

The other five principal components were also plotted in bi-plots. Table 3 shows the contribution of the attributes to each principal component.

Interpretation of the principal components is based on finding which variables are most strongly correlated with each component, that is, which of these numbers is large, the farthest from zero in either direction. The decision as to what values should be considered large is a subjective one and reflects knowledge of the system under evaluation. It was determined that a correlation value was relevant to our study when it was above 0.48, since this value is the largest within each principal component, and most of the variables having this high value are highly correlated, as shown by the components of the correlation matrix. These large correlation values are in boldface in Table 3.

The principal component results can be interpreted with respect to the value deemed to be significant. The first principal component correlated strongly with three of the original attributes. Thus, the first principal component increases with increasing Data storage, Data location, and Delay sensitivity, suggesting that these three attributes vary together.

On the other hand, the coefficients belonging to the second principal component show that the behavior of the feature bandwidth opposes that of the behavior of the features Mobility and Scalability. This reflects the fact that greater mobility is associated with changes in the network topology, which, in turn, increases the fluctuations in communication links and reduces the bandwidth availability. Moreover, since bandwidth is a finite resource, if the number of users connected to the Fog increases the rate at which each user transmits and receives data decreases.

Other features that reveal an inverse relationship are Data Storage and Loss sensitivity, in the fifth principal component, and mobility and loss sensitivity in the seventh principal component.

Finally, the third, fourth, and sixth principal components increase with only one of the values, that is, there is only one variable with a value 0.48 or higher. These variables are Security, Reliability, and Delay sensitivity, respectively. Accordingly, the third, fourth, and sixth principal components can be interpreted as measures of how necessary the use of isolated nodes is to process the application, how quickly failed Fog components should be reestablished, and how sensitive the Fog application is to the delay.

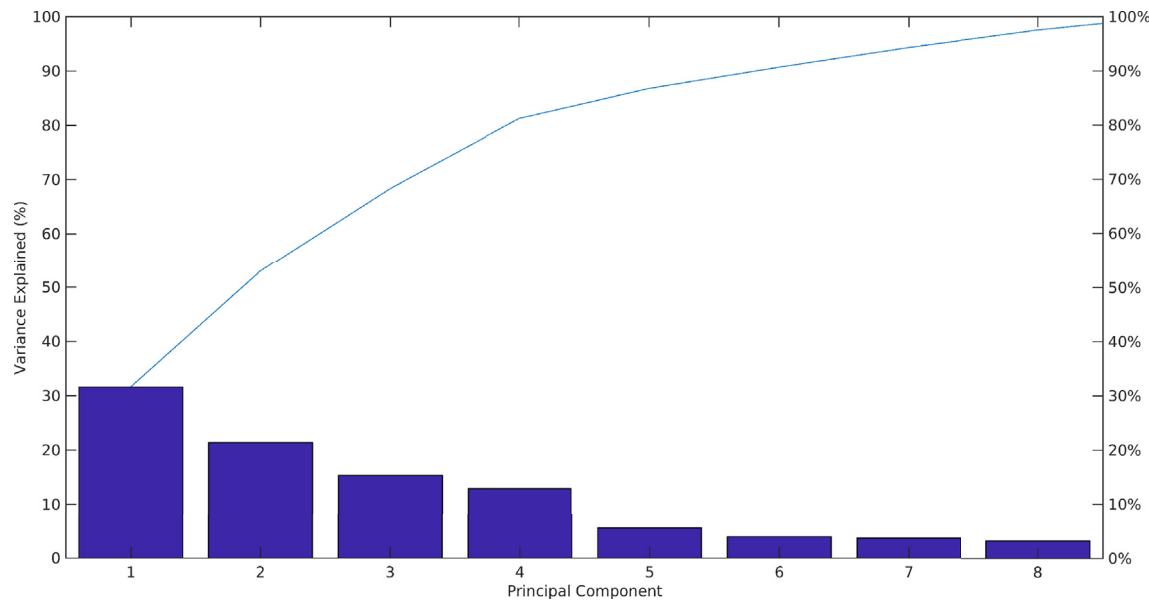


Fig. 5. Scree plot of the percent variability explained by each principal component.

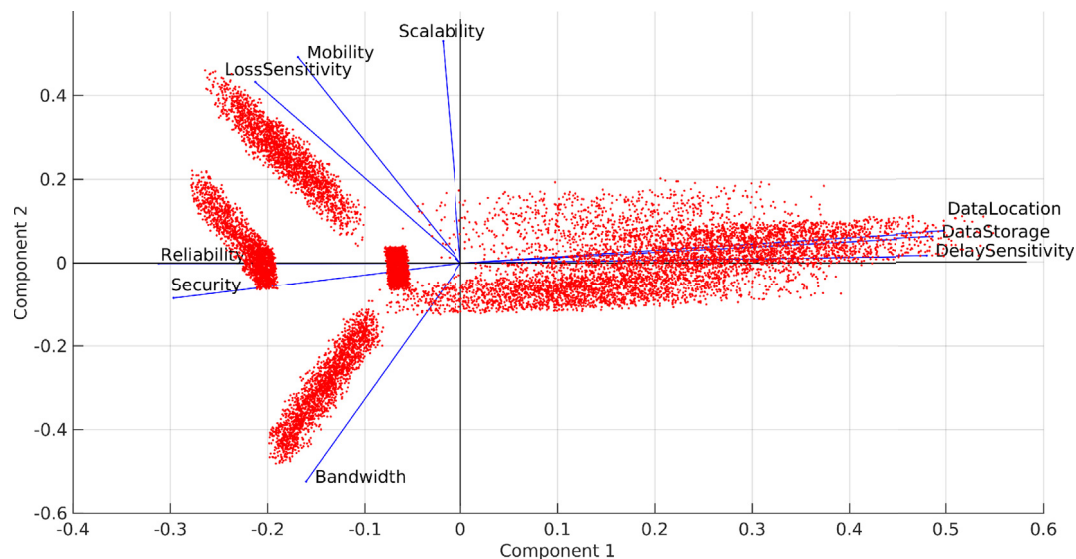


Fig. 6. Orthonormal principal component coefficients for each variable and principal component scored for each observation (principal components 1 and 2).

Table 3

Attribute coefficients for each principal component.

Attribute	Principal Component						
	1	2	3	4	5	6	7
Bandwidth	-0.161	-0.524	-0.449	-0.065	0.079	0.118	0.114
Reliability	-0.312	-0.001	0.040	0.702	0.313	0.295	-0.318
Security	-0.297	-0.085	0.595	0.283	-0.342	-0.286	-0.021
Data Storage	0.486	0.062	-0.099	0.181	0.490	-0.375	-0.389
Data location	0.497	0.075	0.022	0.213	-0.351	-0.283	0.062
Mobility	-0.169	0.491	-0.327	0.350	0.117	-0.207	0.645
Scalability	-0.020	0.530	0.337	-0.366	0.309	0.318	-0.014
Delay sensitivity	0.480	0.016	0.016	0.295	-0.273	0.673	0.134
Loss sensitivity	-0.213	0.432	-0.460	-0.044	-0.481	0.011	-0.544

Based on the analysis described above and considering the semantics of the case study about which variables deserve more attention into the Fog computing environment, redundant attributes such as Data loca-

tion, Data storage, and Mobility have been removed. Thus, six of the original nine attributes were selected and maintained for the stage of classification. These were Delay sensitivity, Scalability, Loss sensitivity,

Security, Reliability, and Bandwidth.

5.3. Classification

Seven classifiers are evaluated for potential adoption: Adaptive neuro-fuzzy inference system from data, using subtractive clustering (ANFIS), Decision Tree (DT) Artificial neural network with 2 hidden layers, trained with the Levenberg-Marquardt backpropagation algorithm, (ANN(1)); Artificial Neural Network with 1 hidden layer, trained with the algorithm Scaled conjugate gradient backpropagation (ANN(2)); Artificial Neural Network with 2 hidden layers, trained with the algorithm Scaled conjugate gradient backpropagation (ANN(3)); K-Nearest Neighbors (KNN), and Support Vector Machine (SVM). These algorithms have different characteristics with respect to noise sensitivity, the speed of learning, and the speed of prediction. For instance, SVMs are known to be very accurate but also sensitive to noise (García et al., 2015). ANNs predict rapidly, but the speed of learning is low, while KNN provides rapid learning, but are considered difficult to interpret (Choose Classifier Options, 2018). These classifiers are typically employed in the context of complex recognition or prediction applications, especially when there is a lack of labeled samples, a lack of time for training and testing, or even a lack of appropriate hardware for timely assessment of the quality of classification models. These same conditions are present in typical Fog computing scenarios, as shown in Fig. 1, in which heterogeneous devices with limited computing and storage capabilities must interoperate in real time to ensure compliance with the QoS requirements of time-sensitive applications. The investigation of data-driven approaches (e.g., based on deep learning models) for constrained processing scenarios (Howard et al., 2017; Sandler et al., 2018; Iandola et al., 2016) as in our target application is left for future work.

5.4. Performance evaluation

This section focuses on two main subtasks of the evaluation process: measurements of the performance and statistical significance of the performance metrics. The hardware configuration used for both the dataset generation and the experiments carried out was: Intel Core i7 processor, 12 GB of RAM, 1 TB of storage and Windows 10 operating system. First, the performance of each classifier is evaluated under ideal conditions, that is, without noise. Each classifier is assessed by measuring its accuracy and efficiency. Then, the performance of each classifier is assessed in the presence of noise. In this case, the robustness of each classifier in two different scenarios is assessed: when noise is introduced into the training set, and when noise is introduced in the testing set. In both cases, with and without noise, Wilcoxon's signed-rank test is employed to determine whether or not the difference in accuracy between two classifiers is statistically significant. Inserting noise in the training data set is designed to evaluate the robustness of the classifier trained by the specific data set. Inserting noise in the test data set aims at assessing the robustness of a classifier in the face of data sets with no evident data relationship, for which data relationships are not so evident.

A stratified 10-Fold Cross-validation (10-FCV) (Kohavi, 1995) protocol was adopted. In this protocol, there are 10 partitions, each with the same proportion of samples belonging to each class. Nine of them are used for training, and the remaining fold is used for testing. This process is repeated 10 times, with each of the 10 folds used exactly once as the testing data. Finally, the 10 results obtained from each one of the test partitions are combined to produce a single average value representing accuracy. Table 4 summarizes the results obtained from the accuracy estimation for each classifier assessed without noise.

Table 4 shows that the average accuracy results for the testing process were close to 100%, except for the ANFIS was only 99.2% accurate. The shortest prediction time was obtained with the DT algorithm, while the KNN took the longest time. One possible explanation is that the DT

Table 4
Accuracy estimation results.

Classification Technique	Testing	
	Average Accuracy (%)	Average Time (s)
ANFIS	99.271	0.048
DT	99.986	0.029
ANN(1)	100	0.032
ANN(2)	100	0.035
ANN(3)	100	0.031
KNN	100	0.073
SVM	100	0.044

algorithm divides the input space, matching the way the attributes were defined and assigned to each CoS by using intervals. The way the synthetic dataset was created may have led to application instances with properties which were easy to model, thus boosting the performance of the evaluated classifiers.

A two-tailed Wilcoxon signed-rank test with a significance level of 0.05 was also applied to the observations obtained from the different classifiers with no attribution of noise. The results revealed that the accuracy level of the classification obtained from the DT, ANN(1), ANN(2), ANN(3), KNN, and SVM were approximately the same. In contrast, observations obtained from the ANFIS revealed statistically significant differences in relation to the observations obtained from the other classifiers.

An attribute noise was introduced into the original dataset to check the effect of noise on the classifiers. Corrupting the data impacts directly on the significance of the attributes for the purpose of classification (García et al., 2015). Moreover, attribute noise includes erroneous attribute values, which is one of the most common types of noise in real-world data (Zhu and Wu, 2004).

Noise is introduced into each partition in a controlled manner, i.e., a certain percentage of the values of each attribute in the dataset were altered. To this end, the steps employed by the authors in (García et al., 2015) were followed. To corrupt each attribute A_i , a certain percentage of the examples in the dataset were chosen, and the A_i value of each was assigned a random uniform value from the domain D_i of the attribute A_i .

Noise was introduced into the training partitions to create a noisy data set from the original, as follows:

1. Noise as a percentage of the original value was introduced into a copy of the full original dataset.
2. The two datasets, the original and the noisy copy, were partitioned into 10 equal folds, i.e., each with the same number of examples of each class.
3. The training partitions are built from the noisy copy, whereas the test partitions were formed from examples from the noise-free dataset.

Noise was introduced into the testing partitions following the same steps described above, except that in Step 3 the testing partitions are built from the noisy copy, while the training partitions were formed from examples from the noise-free data set.

The methodology followed in this subsection presents two differences concerning the methodology described in (García et al., 2015). First, the one proposed here includes the same number of examples of each class in the percentage of values of each attribute in the data set to be corrupted. Second, the introduction of attribute noise was extended to a second simulation scenario in which accuracy and robustness were estimated for each one of the individual classifiers using a clean training set and a testing set with attribute noise. Introducing attribute noise into the training set while maintaining the testing set clean enabled the assessment of the capacity of the classifier to deal with problems in the training stage, such as overfitting. Overfitting occurs when the model is too tightly adjusted to data offering high precision in known

Table 5
Test accuracy and RLA results of classifiers trained with noisy datasets.

	Noise Level (%)	ANFIS	DT	ANN(1)	ANN(2)	ANN(3)	KNN	SVM
Test accuracy results	0	99.271	99.986	100.000	100.000	100.000	100.000	100.000
	10	94.429	99.986	99.911	99.950	99.921	99.800	99.997
	30	84.900	99.950	99.676	99.800	99.691	99.029	99.836
	50	78.564	99.979	90.807	91.370	91.419	99.193	99.600
RLA values results	0	–	–	–	–	–	–	–
	10	0.04878	0.00000	0.00089	0.00050	0.00079	0.00200	0.00003
	30	0.14477	0.00036	0.00324	0.00200	0.00309	0.00971	0.00164
	50	0.20859	0.00007	0.09193	0.08630	0.08581	0.00807	0.00400

cases, but behaving poorly with unseen data. Conversely, introducing attribute noise into the testing set while maintaining the training set clean, enables us to assess the robustness of the trained model.

After introducing noise, the accuracy of classifiers is determined by means of 5 runs of a stratified 10-Fold Cross-Validation (FCV). Hence, a total of 50 runs per dataset, noise type, and level are averaged. Ten partitions make the noise effects more notable, since each partition has a large number of examples (1,400). The robustness of each algorithm is then estimated by using the Relative Loss of Accuracy (RLA) given by Equation (1) is:

$$RLA_{x\%} = \frac{Acc_{0\%} - Acc_{x\%}}{Acc_{0\%}}, \quad (1)$$

where $RLA_{x\%}$ is the relative loss of accuracy at a noise level of $x\%$. $Acc_{0\%}$ is the test accuracy in the original case, that is, with 0% of induced noise, and $Acc_{x\%}$ is the test accuracy with a noise level $x\%$.

Next, the robustness of the classifiers when the noise has been introduced for the two mentioned scenarios will be evaluated.

5.4.1. Classification using a training set with attribute noise and a clean testing set

Table 5 shows the average performance and robustness results for each classification algorithm at each noise level, from 0% to 50%, on training datasets with uniform attribute noise.

As can be observed in Table 5, the DT is the most robust classifier for all noise levels. On the other hand, the ANN(1), ANN(2), and ANN(3) present high robustness for noise levels (10–30%). Conversely, the RLA of classifiers based on neural networks rises linearly to 9% when the noise level is 50%. The least robust classifier is the ANFIS, for which the loss of accuracy increases exponentially as the proportion of noise level rises, to the point that when the noise level is 50%, its RLA is above 21% of that a clean dataset.

Fig. 7 shows the accuracy ratio and testing time when training takes place with both clean datasets, and those disrupted by uniform attribute noise levels of 10%, 30%, and 50%. A marker identifies each classification algorithm, and a different color identifies each noise level. The light-bands indicate the areas of the greatest accuracy or the slowest testing times, and the light-purple intersection of these bands indicates the area where the best results for both accuracy and testing time are found.

The DT algorithm takes only 25 ms for classification with the greatest accuracy for up to 1400 applications simultaneously arriving at the edge, when training has taken place using datasets with a uniform attribute noise level of 50%.

Table 6 presents the results of a two-tailed Wilcoxon signed-rank test (considering a significance level of 0.05) to verify statistical differences between the accuracy results of classifiers trained with noisy datasets. Each cell shows the results of the statistical tests between a single classifier with the others for the four levels of noise (nl). Left ‘←’ and up ‘↑’ arrows indicate the most accurate, while an empty cell refers to ‘no statistical difference between the pairs of classifiers’ in that row and column.

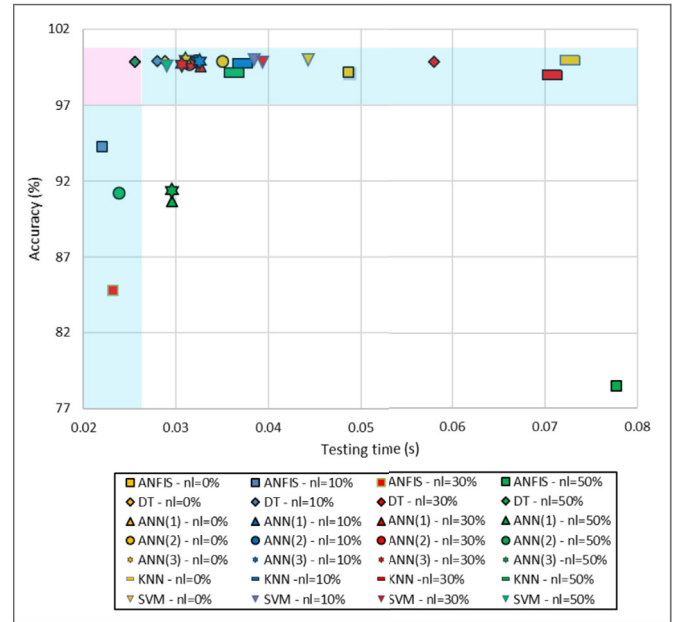


Fig. 7. Accuracy rates concerning the testing time for classifiers trained with both clean and noisy datasets.

Table 6 shows the effect of noise data sets in training. The performance of ANFIS is the least accurate. ANN(1), ANN(2), and SVM produce a similar performance while the DT outperform most of the results of the rest of classifiers when it is trained using datasets with an attribute noise level equal to or greater than 30%.

5.4.2. Classification using a clean training set and a testing set with attribute noise

Table 7 shows the results for average performance and robustness for each classification algorithm at each noise level, from 0% to 50%, from the testing of datasets with uniform attribute noise.

As evinced in Table 7, for all classifiers, accuracy decreases exponentially with an increase in the noise level of the testing dataset. In this situation, the most robust classifiers are ANN(2), DT, ANN(3), ANN(1), and KNN.

Fig. 8 illustrates the results of the classification algorithms when both accuracy and testing time are considered with both clean and noise datasets (levels of 10%, 30%, and 50%). A marker identifies each classification algorithm, and a different color identifies each noise level. The light-blue bands indicate the areas of the greatest accuracy rates or the slowest testing times, and the light-purple intersection of these bands indicates the area where the best results were obtained when considering both accuracy and testing time.

The DT algorithm takes less than 30 ms for classification for with the greatest level of accuracy, up to 1400 applications simultaneously arriving at the edge, when the input is a noise dataset as long as the

Table 6

Statistical test for the accuracy of classifiers trained with noisy datasets. nl denotes the percentage of noise level present in the training dataset.

	nl	ANFIS	DT	ANN(1)	ANN(2)	ANN(3)	KNN	SVM
ANFIS	0	–	↑	↑	↑	↑	↑	↑
	10	–	↑	↑	↑	↑	↑	↑
	30	–	↑	↑	↑	↑	↑	↑
	50	–	↑	–	–	–	↑	↑
DT	0	←	–	–	–	–	–	–
	10	←	–	–	–	–	←	–
	30	←	–	←	←	←	←	←
	50	←	–	←	←	←	←	←
ANN(1)	0	←	–	–	–	–	–	–
	10	←	–	–	–	–	–	–
	30	←	↑	–	–	–	←	–
	50	–	↑	–	–	–	–	–
ANN(2)	0	←	–	–	–	–	–	–
	10	←	–	–	–	–	←	–
	30	←	↑	–	–	–	←	–
	50	–	↑	–	–	–	–	–
ANN(3)	0	←	–	–	–	–	–	–
	10	←	–	–	–	–	–	↑
	30	←	↑	–	–	–	←	↑
	50	–	↑	–	–	–	–	–
KNN	0	←	–	–	–	–	–	–
	10	←	↑	–	↑	–	–	↑
	30	←	↑	↑	↑	↑	–	↑
	50	←	↑	–	–	–	–	↑
SVM	0	←	–	–	–	–	–	–
	10	←	–	–	–	←	←	–
	30	←	↑	–	–	←	←	–
	50	←	↑	–	–	–	←	–

Table 7

Test accuracy and RLA results of classifiers tested with noisy datasets.

	Noise Level (%)	ANFIS	DT	ANN(1)	ANN(2)	ANN(3)	KNN	SVM
Test accuracy results	0	99.271	99.986	100.000	100.000	100.000	100.000	100.000
	10	72.214	85.693	83.570	86.131	84.031	84.093	81.79
	30	41.343	63.114	59.464	63.860	60.467	59.971	55.403
	50	27.414	47.264	45.381	48.564	46.176	44.764	40.799
RLA values results	0	–	–	–	–	–	–	–
	10	0.273	0.143	0.164	0.139	0.160	0.159	0.189
	30	0.584	0.369	0.405	0.361	0.395	0.400	0.446
	50	0.724	0.527	0.546	0.514	0.538	0.552	0.592

noise level does not exceed 10%.

Table 8 presents the results of a two-tailed Wilcoxon signed-rank tests (considering a significance level of 0.05) to verify the statistical differences between the accuracy of the different classifiers when tested with noisy datasets. Each cell shows the result of the statistical test between the pairs of classifiers with different percentages of noise (nl). Left ‘←’ and up ‘↑’ arrows indicate the greatest accuracy while an empty cell refers to “no statistical difference between the pair of classifiers” in that row and column.

The effect of noisy data set in testing is shown in **Table 8**. The performance of ANFIS is the least accurate. Moreover, the DT produces the most accurate classification results in classifications independent of the presence of noise.

5.5. Classification model

The final step of the proposed methodology is the selection of a classifier. The results indicate that the DT was the most accurate and robust classifier.

The decision tree algorithm does not need to assess all the attributes to classify an application since various services have exclusive features. For example, mission-critical applications are the only ones for which

reliability takes on a “critical” value. Therefore, assessing certain features makes classification a more efficient process. This is an attractive characteristic which makes Decision Tree an ideal algorithm for the classification of applications in Fog computing. Moreover, the Decision Tree algorithm is easy to interpret, fast for fitting and prediction, and does not use much memory. Given these characteristics, the Decision Tree algorithm can be run by devices such as routers, switches, and servers, located on the first Fog layer of the reference architecture introduced by the OpenFog Consortium in ([OpenFog Reference Architecture, 2017](#)). After classification, the output of the classifier serves as input for the scheduler, also located at the first Fog layer, which decides where the application should be processed.

6. Conclusions

This paper has introduced the use of ML classification algorithms as a tool for QoS-aware resource management in Fog computing. First, potential Fog computing applications are grouped in seven CoS according to their QoS requirements. A synthetic database of Fog applications is built from the definition of the intervals that each QoS requirement relevant for a specific Class. At this point, the dataset is pre-processed to convert prior useless data into new data that can be used by ML

Table 8

Statistical test for the accuracy of classifiers tested with noisy datasets. nl denotes the percentage of noise level present in the testing dataset.

	nl	ANFIS	DT	ANN(1)	ANN(2)	ANN(3)	KNN	SVM
ANFIS	0	–	↑	↑	↑	↑	↑	↑
	10	–	↑	↑	↑	↑	↑	↑
	30	–	↑	↑	↑	↑	↑	↑
	50	–	↑	↑	↑	↑	↑	↑
DT	0	←	–	–	–	–	–	–
	10	←	–	←	–	←	←	←
	30	←	–	←	–	←	←	←
	50	←	–	–	–	←	←	←
ANN(1)	0	←	–	–	–	–	–	–
	10	←	↑	–	↑	–	–	←
	30	←	↑	–	↑	–	–	←
	50	←	–	–	↑	–	–	←
ANN(2)	0	←	–	–	–	–	–	–
	10	←	–	←	–	←	←	←
	30	←	–	←	–	←	←	←
	50	←	–	←	–	←	←	←
ANN(3)	0	←	–	–	–	–	–	–
	10	←	↑	–	↑	–	–	←
	30	←	↑	–	↑	–	–	←
	50	←	–	–	–	–	–	←
KNN	0	←	–	–	–	–	–	–
	10	←	↑	–	↑	–	–	←
	30	←	↑	–	↑	–	–	←
	50	←	↑	–	↑	–	–	←
SVM	0	←	–	–	–	–	–	–
	10	←	↑	↑	↑	↑	↑	–
	30	←	↑	↑	↑	↑	↑	–
	50	←	↑	↑	↑	↑	↑	–

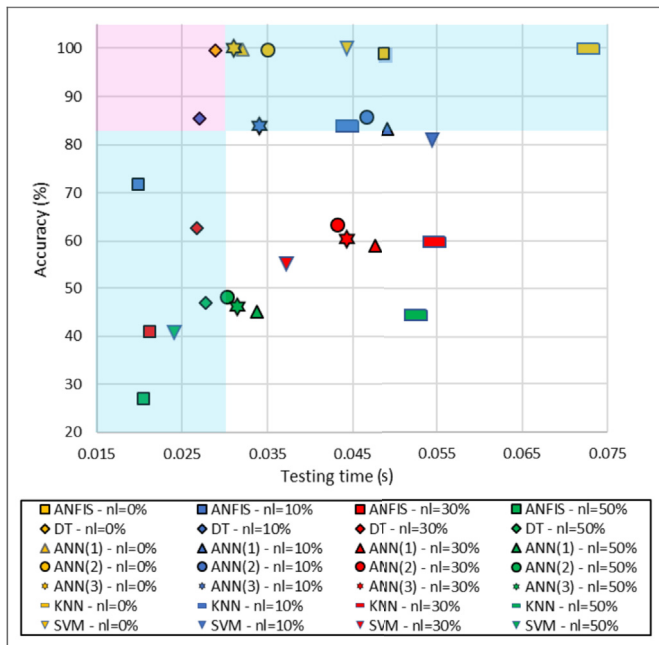


Fig. 8. Accuracy rates concerning the testing time for classifiers tested with both clean and noisy datasets.

techniques. Next, a set of popular ML algorithms is selected and puts through the training and testing processes, using the examples in the synthetic database to measure the degree of accuracy and efficiency in their prediction of the CoS to which the application belongs. For this, the synthetic database is contaminated with three different levels of attribute noise. For each noise level, the classifier conducts training and testing to measure the degree of robustness.

This ML-based classification methodology allows the implementation of CoS to manage the traffic in Fog, which constitutes a first step in the definition of QoS provisioning mechanisms in the C2T continuum. Moreover, the integration of IoT, Fog, and Cloud requires efficient management strategies capable of facilitating resource management tasks such as scheduling, allocation, and federation. The classification of Fog applications resolves all these issues; moreover, it is easy to implement into existing devices, and represents a new way of self-adaptive and cognitive network mechanisms (Batista et al., 2007, 2008; Mesodiakaki et al., 2014; Baggio et al., 2019; Merchant et al., 2018), which are fundamental for the deployment of autonomic Fog systems in environments with limited availability of processing resources.

For future work, an ML-based classification algorithm will be integrated into the Fog network scheduler, thus enabling the network scheduler to prioritize processing requests. It will also allow more delay sensitive demands to be satisfactorily fulfilled. We also suggest the inclusion of other feature selection methods such as Relief-F, CFS, MCFS, and the Student's t-test in the pre-processing stage for future classification studies in the context of Fog computing. Moreover, we recommend a broad study on the distribution of noise for data analysis oriented to parameters related to Cloud/Fog/edge applications as well as network traffic.

Author-contribution

Judy Guevara – the manuscript is part of her Ph.D. thesis. She programmed, generated the data, co-wrote and revised the manuscript.

Ricardo Torres – advised on machine learning techniques, co-wrote and revised the manuscript. Nelson Fonseca – thesis supervisor, advised on Fog and networking content, co-wrote and revised the manuscript.

Declaration of competing interest

None.

Acknowledgments

This work was supported by the Brazilian National Research Agency CNPq and the Academy of Sciences for the Developing World (TWAS), under process 190172/2014-2 of the CNPq-TWAS program. The authors are also grateful to CAPES (grant No. 88881.145912/2017-01), CNPq (grant No. 307560/2016-3), FAPESP (grants Nos. 2014/12236-1, 2015/24494-8, 2016/50250-1, and 2017/20945-0), and the FAPESP-Microsoft Virtual Institute (grants Nos. 2013/50155-0, 2013/50169-1, and 2014/50715-9). The authors would like to thank reviewers constructive comments.

References

- Aazam, M., St-Hilaire, M., Lung, C., Lambadaris, I., 2016. Mefore: qoe based resource estimation at fog to enhance qos in iot. In: 2016 23rd International Conference on Telecommunications (ICT), pp. 1–5, <https://doi.org/10.1109/ICT.2016.7500362>.
- Alhamad, M., Dillon, T., Chang, E., 2010. Conceptual sla framework for cloud computing. In: 4th IEEE International Conference on Digital Ecosystems and Technologies, pp. 606–610, <https://doi.org/10.1109/DEST.2010.5610586>.
- Ali, N.A., Taha, A.M., Hassanein, H.S., 2013. Quality of service in 3gpp r12 lte-advanced. *IEEE Commun. Mag.* 51 (8), 103–109, <https://doi.org/10.1109/MCOM.2013.6576346>.
- Ali, M.A., Esmailpour, A., Nasser, N., 2017. Traffic density based adaptive QoS classes mapping for integrated LTE-WiMAX 5G networks. In: 2017 IEEE International Conference on Communications (ICC), pp. 1–7.
- Alkassab, N., Huang, C.T., Chen, Y., Choi, B.Y., Song, S., 2017. Benefits and schemes of prefetching from cloud to fog networks. In: 2017 IEEE 6th International Conference on Cloud Networking (CloudNet), pp. 1–5.
- Arkian, H.R., Diyanat, A., Pourkhalili, A., 2017. MIST: fog-based data analytics scheme with cost-efficient resource provisioning for IoT crowdsensing applications. *J. Netw. Comput. Appl.* 82, 152–165.
- Baggio, G., Bassoli, R., Granelli, F., 2019. Cognitive software-defined networking using fuzzy cognitive maps. *IEEE Trans. Cogn. Comm. Network.* 5 (3), 517–539, <https://doi.org/10.1109/TCCN.2019.2920593>.
- Batista, D.M., Fonseca, N.L.S.d., 2010. A survey of self-adaptive grids. *IEEE Commun. Mag.* 48 (7), 94–100.
- Batista, D.M., da Fonseca, N.L.S., 2011. Robust scheduler for grid networks under uncertainties of both application demands and resource availability. *Comput. Network.* 55 (1), 3–19.
- Batista, D.M., Fonseca, N.L.S.d., Granelli, F., Kliazovich, D., 2007. Self-adjusting grid networks. In: 2007 IEEE International Conference on Communications, pp. 1–5.
- Batista, D.M., da Fonseca, N.L.S., Miyazawa, F.K., Granelli, F., 2008. Self-adjustment of resource allocation for grid applications. *Comput. Network.* 52 (9), 1762–1781.
- Bittencourt, L.F., Madeira, E.R.M., Fonseca, N.L.S.d., 2012. Scheduling in hybrid clouds. *IEEE Commun. Mag.* 50 (9), 42–47.
- Bittencourt, L.F., Diaz-Montes, J., Buyya, R., Rana, O.F., Parashar, M., 2017. Mobility-aware application scheduling in fog computing. *IEEE Cloud Comput.* 4 (2), 26–35.
- Bhmer, M., Hecht, B., Schning, J., Krger, A., Bauer, G., 2011. Falling asleep with angry birds, facebook and kindle: a large scale study on mobile application usage. In: Proceedings of the 13th International Conference on Human Computer Interaction with Mobile Devices and Services, MobileHCI 11. ACM, New York, NY, USA, pp. 47–56, <https://doi.org/10.1145/2037373.2037383>.
- Bonomi, F., Milito, R., Zhu, J., Addepalli, S., 2012. Fog computing and its role in the Internet of Things. In: Proceedings of the First Edition of the MCC Workshop on Mobile Cloud Computing, MCC 12. ACM, New York, NY, USA, pp. 13–16.
- Byers, C.C., 2017. Architectural imperatives for fog computing: use cases, requirements, and architectural techniques for fog-enabled IoT networks. *IEEE Commun. Mag.* 55 (8), 14–20, <https://doi.org/10.1109/MCOM.2017.1600885>.
- Cai, D., Zhang, C., He, X., 2010. Unsupervised feature selection for multi-cluster data. In: Proceedings of the 16th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD 10. ACM, New York, NY, USA, pp. 333–342.
- Callado, A., Kamiński, C., Szabo, G., Gero, B.P., Kelner, J., Fernandes, S., Sadok, D., 2009. A survey on Internet traffic identification. *IEEE Commun. Surv. Tutor.* 11 (3), 37–52.
- Cardellini, V., Grassi, V., Presti, F.L., Nardelli, M., 2015. On qos-aware scheduling of data stream applications over fog computing infrastructures. In: 2015 IEEE Symposium on Computers and Communication. ISCC, pp. 271–276, <https://doi.org/10.1109/ISCC.2015.7405527>.
- Chekired, D.A., Khoukhi, L., Moutaf, H.T., 2018. Industrial iot data scheduling based on hierarchical fog computing: a key for enabling smart factory. *IEEE Trans. Industr. Inform.* 14 (10), 4590–4602, <https://doi.org/10.1109/TII.2018.2843802>.
- Choose Classifier Options - MATLAB & Simulink, 2018. Available: <https://www.mathworks.com/help/stats/choose-a-classifier.html>. (Accessed 21 May 2018).
- Cohen, R., Fonseca, N.L.S., Zukerman, M., 1998. Traffic Management and Control, Multimedia Communication Network: Technologies and Services. Artech House, Norwood, MA, USA.
- Deng, R., Lu, R., Lai, C., Luan, T.H., Liang, H., 2016. Optimal workload allocation in fog-cloud computing toward balanced delay and power consumption. *IEEE Intern. Things J.* 3 (6), 1171–1181.
- Emekaroha, V.C., Brandic, I., Maurer, M., Dustdar, S., 2010. Low level metrics to high level slas - lom2his framework: bridging the gap between monitored metrics and sla parameters in cloud environments. In: 2010 International Conference on High Performance Computing Simulation, pp. 48–54, <https://doi.org/10.1109/HPCS.2010.5547150>.
- Emekaroha, V.C., Ferreto, T.C., Netto, M.A.S., Brandic, I., De Rose, C.A.F., 2012. Casvid: application level monitoring for sla violation detection in clouds. In: 2012 IEEE 36th Annual Computer Software and Applications Conference, pp. 499–508, <https://doi.org/10.1109/COMPSAC.2012.68>.
- Finsterbush, M., Richter, C., Rocha, E., Muller, J., Hanssger, K., 2014. A survey of payload-based traffic classification approaches. *IEEE Commun. Surv. Tutor.* 16 (2), 1135–1156.
- Fonseca, N.L.S.d., Boutaba, R., 2015. Cloud Services, Networking, and Management. John Wiley & Sons.
- Garca, S., Luengo, J., Herrera, F., 2015. Dealing with noisy data. In: Data Preprocessing in Data Mining. Springer International Publishing, Cham, pp. 107–145.
- Genev, T.A.L., Bittencourt, L.F., Fonseca, N.L.S.d., Madeira, E.R.M., 2019. Estimation of the available bandwidth in inter-cloud links for task scheduling in hybrid clouds. *IEEE Trans. Cloud Comput.* 7 (1), 62–74.
- Guyon, I., Weston, J., Barnhill, S., Vapnik, V., 2002. Gene selection for cancer classification using support vector machines. *Mach. Learn.* 46 (1), 389–422.
- He, X., Wang, K., Huang, H., Miyazaki, T., Wang, Y., Sun, Y., 2018. Qoe-driven joint resource allocation for content delivery in fog computing environment. In: 2018 IEEE International Conference on Communications (ICC), pp. 1–6, <https://doi.org/10.1109/ICC.2018.8422843>.
- Hobfeld, T., Schatz, R., Varela, M., Timmerer, C., 2012. Challenges of qoe management for cloud applications. *IEEE Commun. Mag.* 50 (4), 28–36, <https://doi.org/10.1109/MCOM.2012.6178831>.
- Hu, P., Dhelim, S., Ning, H., Qiu, T., 2017. Survey on fog computing: architecture, key technologies, applications and open issues. *J. Netw. Comput. Appl.* 98, 27–42.
- Huber, P., 1981. J. Wiley, W. InterScience, Robust Statistics. Wiley, New York.
- Sandler, M., Howard, A., Zhu A., Zhmoginov, A., Chen, L.-C., 2018. MobileNetV2: Inverted Residuals and Linear Bottlenecks, arXiv:1801.04381 [cs]ArXiv: 1801.04381.
- Iandola, F.N., Han, S., Moskewicz, M.W., Ashraf, K., Dally, W.J., Keutzer, K., 2016. SqueezeNet: AlexNet-Level Accuracy with 50x Fewer Parameters and <0.5mb Model Size, arXiv:1602.07360 [cs]ArXiv: 1602.07360.
- Howard, A. G, Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., Adam, H., 2017. MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications, arXiv:1704.04861 [cs]ArXiv: 1704.04861.
- Jain, R., Jain, R.K., Jain, 1991. The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling, Edio, 1 Edition. Wiley, New York.
- Khan, S., Parkinson, S., Qin, Y., 2017. Fog computing security: a review of current applications and security solutions. *J. Cloud Comput.* 6 (1), 19, <https://doi.org/10.1186/s13677-017-0090-3>.
- Kliazovich, D., Pecero, J.E., Tchernykh, A., Bouvry, P., Khan, S.U., Zomaya, A.Y., 2016. CA-DAG: Modeling communication-aware applications for scheduling in cloud computing. *J. Grid Comput.* 14 (1), 23–39, <https://doi.org/10.1007/s10723-015-9337-8>.
- Kohavi, R., 1995. A study of cross-validation and bootstrap for accuracy estimation and model selection. In: Proceedings of the 14th International Joint Conference on Artificial Intelligence, vol. 2. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, pp. 1137–1143. IJCAI95.
- Krauter, K., Buyya, R., Maheswaran, M., 2002. A taxonomy and survey of grid resource management systems for distributed computing. *Software Pract. Ex.* 32 (2), 135–164, <https://doi.org/10.1002/spe.432>.
- Kumari, A., Tanwar, S., Tyagi, S., Kumar, N., Parizi, R.M., Choo, K.-K.R., 2019. Fog data analytics: a taxonomy and process model. *J. Netw. Comput. Appl.* 128, 90–104.
- Kurose, J.F., Ross, K.W., 2012. Computer Networking: A Top-Down Approach, sixth ed. Pearson. sixth ed..
- Liu, H., Motoda, H., 2007. In: Computational Methods of Feature Selection (Chapman & Hall/Crc Data Mining and Knowledge Discovery Series). Chapman & Hall/CRC.
- Mahmud, R., Srirama, S.N., Ramamohanarao, K., Buyya, R., 2019. Quality of experience (qoe)-aware placement of applications in fog computing environments. *J. Parallel Distr. Comput.* 132, 190–203, <https://doi.org/10.1016/j.jpdc.2018.03.004>, <http://www.sciencedirect.com/science/article/pii/S0743731518301771>.
- McGregor, A., Hall, M., Lorier, P., Brunskill, J., 2004. Flow clustering using machine learning techniques. In: Barakat, C., Pratt, I. (Eds.), Passive and Active Network Measurement, Lecture Notes in Computer Science. Springer Berlin Heidelberg, pp. 205–214.
- Mell, P., Grance, T., Sep. 2011. The NIST Definition of Cloud Computing. Tech. Rep. NIST Special Publication (SP) 800-145. National Institute of Standards and Technology.
- Merchant, K., Revay, S., Stantchev, G., Noursain, B., 2018. Deep learning for rf device fingerprinting in cognitive communication networks. *IEEE J. Select. Top. Sign. Proc.* 12 (1), 160–167, <https://doi.org/10.1109/JSTSP.2018.2796446>.
- Mesodiakaki, A., Adelantado, F., Alonso, L., Verikoukis, C.V., 2014. Energy-efficient user association in cognitive heterogeneous networks. *IEEE Commun. Mag.* 52 (7), 22–29, <https://doi.org/10.1109/MCOM.2014.6852079>.
- Naqa, I.E., Li, R., Murphy, M.J. (Eds.), 2015. Machine Learning in Radiation Oncology: Theory and Applications. Springer International Publishing.
- OpenFog Reference Architecture: OpenFog Consortium, 2017. Available: <https://www.openfogconsortium.org/ra/>. (Accessed 24 May 2017).
- Pearson, K., 1901. On lines and planes of closest fit to systems of points in space. *Phil. Mag.* 2 (11), 559–572.

- Mahmud, R., Buyya, R., 2016. Fog computing: A taxonomy, survey and future directions, CoRR abs/1611.05539. arXiv:1611.05539. URL <http://arxiv.org/abs/1611.05539>.
- Roffo, G., 2016. Feature Selection Library (MATLAB Toolbox), arXiv:1607.01327 [cs]ArXiv: 1607.01327.
- Shao, Y., Li, C., Fu, Z., Jia, L., Luo, Y., 2019. Cost-effective replication management and scheduling in edge computing. *J. Netw. Comput. Appl.* 129, 46–61.
- Skarlat, O., Nardelli, M., Schulte, S., Dustdar, S., 2017. Towards qos-aware fog service placement. In: 2017 IEEE 1st International Conference on Fog and Edge Computing (ICFEC), pp. 89–96, <https://doi.org/10.1109/ICFEC.2017.12>.
- Tan, P.-N., Steinbach, M., Kumar, V., 2005. *Introduction to Data Mining*, first ed. Addison-Wesley Longman Publishing Co., Inc., Boston, MA, USA.
- Tsai, C., Rodrigues, J.J.P.C., 2014. Metaheuristic scheduling for cloud: a survey. *IEEE Syst. J.* 8 (1), 279–291, <https://doi.org/10.1109/JSYST.2013.2256731>.
- Wang, S., Urgaonkar, R., He, T., Chan, K., Zafer, M., Leung, K.K., 2017. Dynamic service placement for mobile micro-clouds with predicted future costs. *IEEE Trans. Parallel Distr. Syst.* 28 (4), 1002–1016, <https://doi.org/10.1109/TPDS.2016.2604814>.
- Wang, T., Liang, Y., Jia, W., Arif, M., Liu, A., Xie, M., 2019. Coupling resource management based on fog computing in smart city systems. *J. Netw. Comput. Appl.* 135, 11–19.
- Wu, L., Garg, S.K., Buyya, R., Chen, C., Versteeg, S., 2013. Automated sla negotiation framework for cloud computing. In: 2013 13th IEEE/ACM International Symposium on Cluster, Cloud, and Grid Computing, pp. 235–244, <https://doi.org/10.1109/CCGrid.2013.64>.
- Wu, F., Wu, Q., Tan, Y., 2015. Workflow scheduling in cloud: a survey. *J. Supercomput.* 71 (9), 3373–3418, <https://doi.org/10.1007/s11227-015-1438-4>.
- Xu, Jin, Lam, A.Y.S., Li, V.O.K., 2011. Chemical reaction optimization for task scheduling in grid computing. *IEEE Trans. Parallel Distr. Syst.* 22 (10), 1624–1631, <https://doi.org/10.1109/TPDS.2011.35>.
- Yang, S., 2017. Iot stream processing and analytics in the fog. *IEEE Commun. Mag.* 55 (8), 21–27, <https://doi.org/10.1109/MCOM.2017.1600840>.
- Zhong, S., Khoshgoftaar, T.M., Seliya, N., 2004. Analyzing software measurement data with clustering techniques. *IEEE Intell. Syst.* 19 (2), 20–27.
- Zhu, X., Wu, X., 2004. Class noise vs. Attribute noise: a quantitative study. *Artif. Intell. Rev.* 22 (3), 177–210.



Judy C. Guevara received her degree in control engineering (2009) and M.Sc. degree in Information and Communication Sciences (2012) from the Universidad Distrital Francisco José de Caldas, Bogotá, Colombia, and is currently working toward the Ph.D. degree at the Institute of Computing, State University of Campinas (Unicamp), Brazil. Her awards include the Young Researchers and Innovators “Virginia Gutiérrez de Pineda” fellowship (2010), supported by the Colombian Administrative Department of Science, Technology and Inno-



vation, COLCIENCIAS; and the CNPq-TWAS Postgraduate Fellowship (2014). Her research interests focus on resource management and scheduling in Cloud and Fog computing networks.

Ricardo da S. Torres is Professor in Visual Computing at the Norwegian University of Science and Technology (NTNU). He used to hold a position as a Professor at the University of Campinas, Brazil (2005 - 2019). Dr. Torres received a B.Sc. in Computer Engineering from the University of Campinas, Brazil, in 2000 and his Ph.D. degree in Computer Science at the same university in 2004. Dr. Torres has been developing multidisciplinary eScience research projects involving Multimedia Analysis, Multimedia Retrieval, Machine Learning, Databases, Information Visualisation, and Digital Libraries. Dr. Torres is author/co-author of more than 200 articles in refereed journals and conferences and serves as a PC member for several international and national conferences. Currently, he has been serving as Senior Associate Editor of the IEEE Signal Processing Letters and Associate Editor of the Pattern Recognition Letters.



Nelson L. S. da Fonseca received the Ph.D. degree in computer engineering from the University of Southern California, Los Angeles, CA, USA, in 1994. He is currently a Full Professor with the Institute of Computing, State University of Campinas, Campinas, Brazil. He has authored or coauthored over 400 papers and has supervised over 60 graduate students. Prof. Fonseca is currently the Vice President Technical and Educational Activities of the IEEE Communications Society (ComSoc). He served as the ComSoc VicePresident Publications, Vice President Member Relations, Director of Conference Development, Director of Latin America Region, and Director of On-Line Services. He is the Past Editor-in-Chief of IEEE Communications Surveys and Tutorials. He is Senior Editor of the IEEE Communications Magazine, an Editorial Board Member of Computer Networks, Peer-to-Peer Networking and Applications. He was a recipient of the 2012 IEEE Communications Society (ComSoc) Joseph LoCicero Award for Exemplary Service to Publications, the Medal of the Chancellor of the University of Pisa, in 2007, and the Elsevier Computer Network Journal Editor of Year 2001 Award.